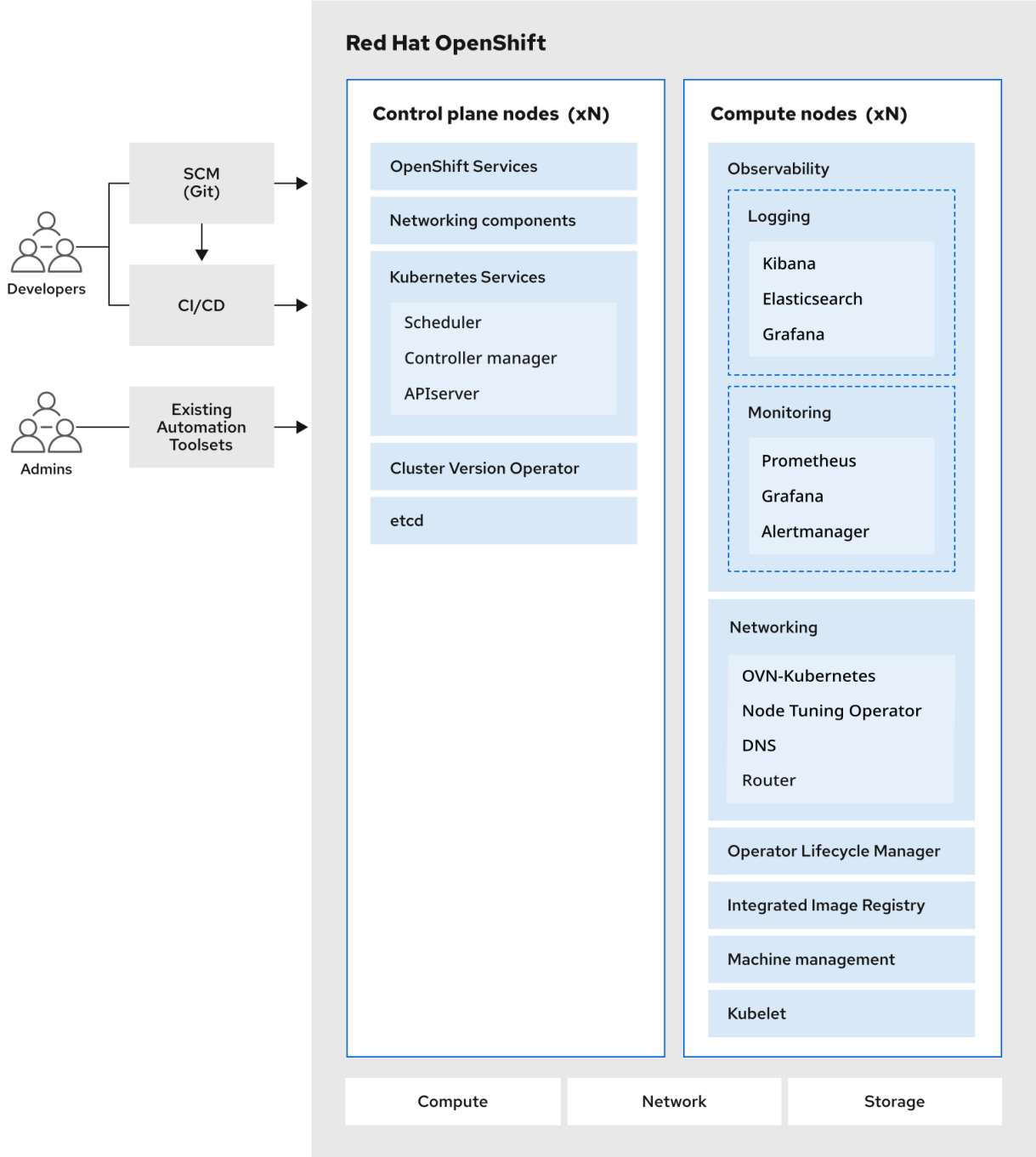


İçindekiler

OpenShift Container Platform (OCP) nedir?	2
Kubernetes?	3
Container ne demek?	4
OpenShift'in Kubernetes'ten farkı ne?	5
OpenShift Mimarisi (High Level).....	5
Control Plane (Master Node'lar)	6
Worker Node'lar.....	7
Infrastructure Node'lar	7
OpenShift Node Rollerı	7
Nerede çalışır?	7
Ne zaman OpenShift seçilir?	8
OpenShift'i Nasıl Çalıştırabiliriz?	8
OpenShift Arayüzü.....	9
OpenShift Giriş Sonrası Ana Ekran (Overview)	9
Sol Navigasyon Menüsü (Ana Menü)	10
Developer / Administrator Perspektif Seçimi.....	11
Project (Namespace) Seçimi	12
Topology Görünümü (Uygulama Haritası)	13
Workloads → Pods	14
Pod Detayı & Log Görünümü.....	14
Log Sekmesinin Üst Bölümü.....	15
Pod Detayı & Log Örneği	16
Deployment Yönetimi.....	16
Networking → Services	17
Networking → Routes (URL).....	18
Storage → PVC	18
Observe → Monitoring.....	18
Administration → Nodes	19
Cluster Operators	19
Kaynaklar.....	20

OpenShift Container Platform (OCP) nedir?



525-OpenShift-arch-012025

OpenShift Container, **Red Hat tarafından geliştirilen, Kubernetes tabanlı bir konteyner platformudur.** Kullanıcıların kapsayıcı tabanlı uygulamaları ve iş yüklerini çalıştırmasını sağlar. AngularJS ve Go programlama dilinde yazılmıştır.

OpenShift Container Platform'un temeli Kubernetes'e dayanmaktadır ve bu nedenle aynı teknolojiyi paylaşmaktadır.

OpenShift = Kubernetes + güvenlik + geliştirici araçları + kurumsal kolaylıklar

OpenShift, uygulamaları **container (konteyner)** içinde:

- geliştirmek
 - test etmek
 - deploy etmek
 - ölçeklemek
- için kullanılan **enterprise seviyede** bir platformdur.

Altyapıda **Kubernetes** çalışır ama OpenShift:

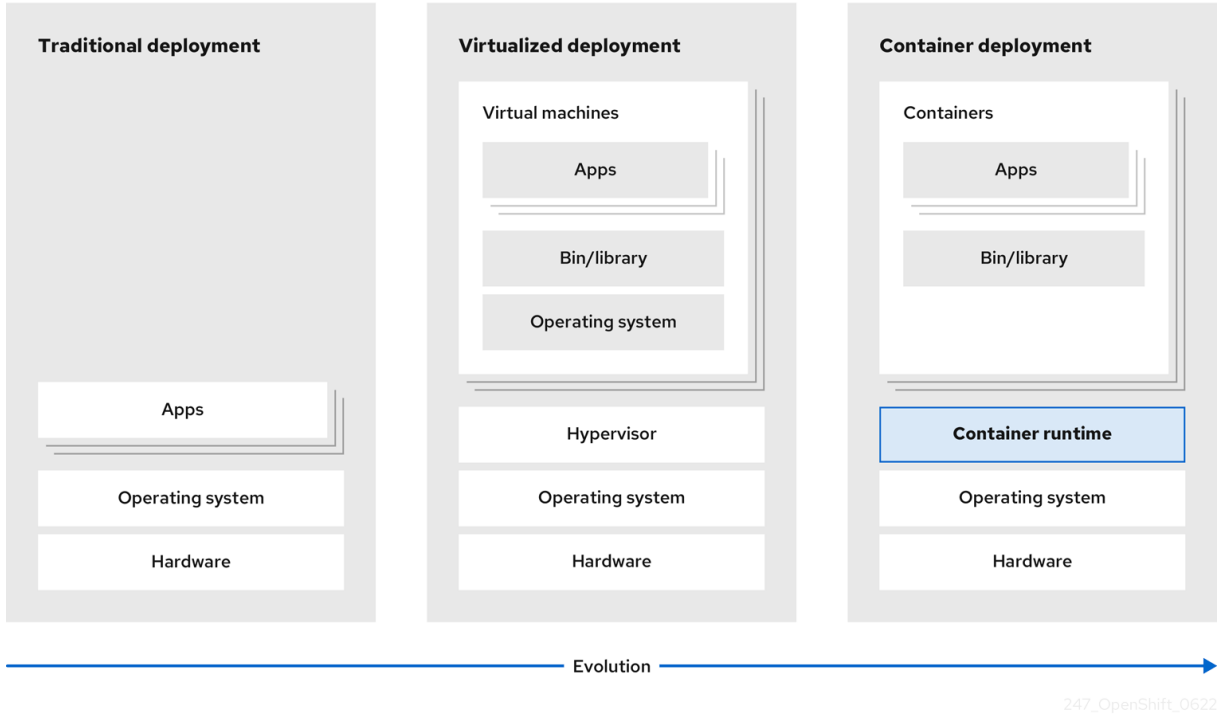
- kurulumu,
- güvenliği,
- CI/CD entegrasyonlarını
çok daha “hazır” hale getirir.

Kubernetes?

Kubernetes, Google tarafından geliştirilen açık kaynaklı bir konteyner düzenleme aracıdır. Kubernetes kullanarak konteyner tabanlı iş yüklerini çalıştırabilir ve yönetebilirsiniz.

Geleneksel olarak, uygulamalar tek bir işletim sistemi üzerinde dağıtılıyordu. Sanallaştırma ile fiziksel sunucuyu birden fazla sanal sunucuya bölebilirsiniz. Paylaşılan kaynaklar üzerinde sanal örneklerle çalışmak verimlilik ve ölçeklenebilirlik açısından en uygun yöntem değildir. Çünkü bir sanal makine (VM), fiziksel bir makine kadar kaynak tüketir; bu nedenle, bir VM'ye CPU, RAM ve depolama gibi kaynaklar sağlamak pahalı olabilir. Ayrıca, paylaşılan kaynaklar üzerinde sanal örnek kullanımı nedeniyle uygulamanızın performansında düşüş görebilirsiniz.

Klasik dağıtımlar için konteyner teknolojilerinin evrimi



Bu sorunu çözmek için, uygulamaları kapsayıcılaştırılmış bir ortamda ayıran kapsayıcılaştırma teknolojilerini kullanabilirsiniz. Bir sanal makineye benzer şekilde, bir kapsayıcının kendi dosya sistemi, sanal işlemcisi, belleği, işlem alanı, bağımlılıkları ve daha fazlası vardır. Kapsayıcılar, altta yatan altyapıdan bağımsızdır ve bulutlar ve işletim sistemi dağıtımları arasında taşınabilir. Kapsayıcılar, tam özellikli bir işletim sisteminden doğal olarak çok daha hafiftir ve işletim sistemi çekirdeğinde çalışan hafif, izole edilmiş süreçlerdir. Sanal makinelerin önyüklemesi daha yavaştır ve fiziksel donanımın bir soyutlamasıdır. Sanal makineler, bir hipervizör yardımıyla tek bir makinede çalışır.

Kubernetes kullanarak aşağıdaki işlemleri gerçekleştirebilirsiniz:

- Kaynakların paylaşılması
- Birden fazla sunucuda konteynerlerin düzenlenmesi
- Yeni donanım yapılandırmalarının yüklenmesi
- Sağlık kontrolleri ve kendi kendini onaran uygulamalar çalıştırılıyor.
- Konteynerleştirilmiş uygulamaların ölçeklendirilmesi

Container ne demek?

Container, bir uygulamanın çalışması için gereken tüm bileşenlerin (kod, kütüphaneler, bağımlılıklar) bir araya getirildiği hafif bir sanallaştırma yöntemidir. Konteynerler, çekirdek içermeyen küçük, özel Linux işletim sistemleri kullanır.

Linux tabanlı olmaları, konteynerlerin hızlı inovasyon sağlayan açık kaynak geliştirme modelinin tüm avantajlarından yararlanmasını sağlar.

Container:

- Uygulama + bağımlılıkları
- Hafif, hızlı ve taşınabilir

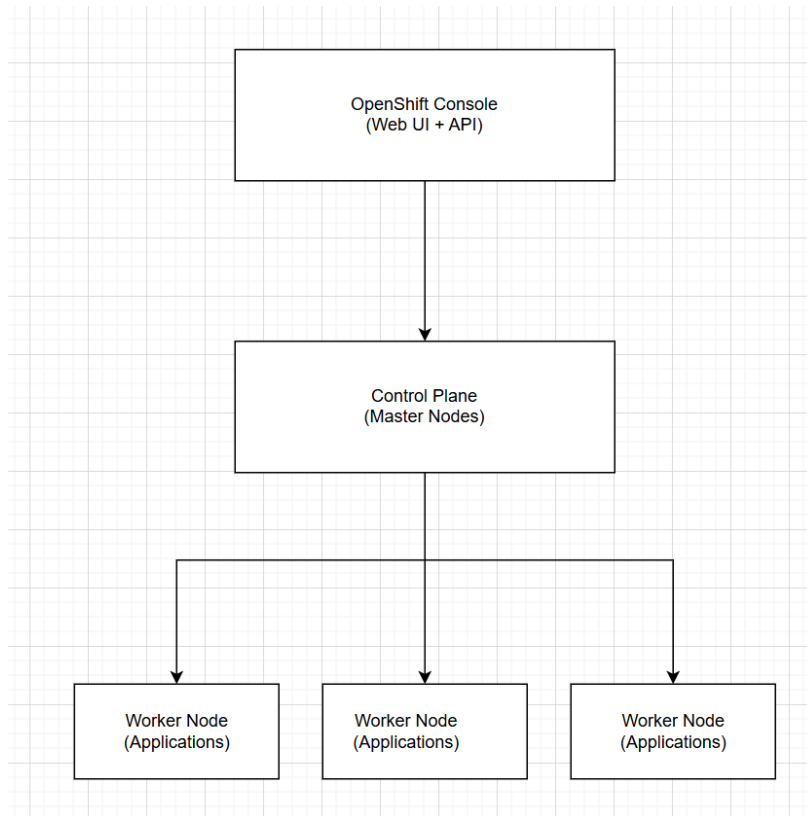
Docker en bilinen container teknolojisidir. OpenShift ise bu container'ları **Kubernetes ile yönetir**.

OpenShift'in Kubernetes'ten farkı ne?

Özellik	Kubernetes	OpenShift
Kurulum	Zor	Daha hazır
Güvenlik	Manuel	Varsayılan güvenli
CI/CD	Harici	Dahili (Pipelines)
Registry	Harici	Dahili
UI	Sınırlı	Güçlü Web Console
Enterprise Support	Yok	Red Hat desteği

OpenShift Mimarisi (High Level)

OpenShift mimarisi temelde **Kubernetes Control Plane + Worker Node'lar** üzerine kuruludur ama üstüne Red Hat'ın ekledikleri vardır.



Control Plane (Master Node'lar)

OpenShift genelde **3 veya 5 master node** ile kurulur (HA için). Master nodes, platform yönetimini ve kontrolünü sağlar.

Temel bileşenler:

API Server

- Tüm isteklerin giriş kapısı
- oc, Web Console, CI/CD hepsi buraya gelir

etcd

- Cluster'ın **beyni**
- Config, secret, state burada tutulur
- Dağıtık key-value database

⚠ Kritik → yedeği çok önemli

Scheduler

- Pod hangi worker node'da çalışacak karar verir
- CPU, RAM, affinity kurallarına bakar

Controller Manager

- İstenen durum ≠ mevcut durum ise düzeltir
- Örnek:
 - Pod öldü → yenisini başlat
 - Replica 3 → 2 kaldı → tamamla

Worker Node'lar

Worker nodes, container workload'larını çalıştırır.

Worker Node bileşenleri:

kubelet

- Master'dan komut alır
- Pod'ları başlatır / durdurur

Container Runtime

- OpenShift 4 → **CRI-O**
- Docker yok ❌

Pod'lar

- En küçük deploy birimi
- İçinde:
 - 1 veya daha fazla container
 - Shared network & volume

Infrastructure Node'lar

Infrastructure nodes, platform servislerini barındırır.

OpenShift Node Roller

Rol	Görev
Master	Control Plane
Worker	Application
Infra	Router, Registry, Monitoring

Nerede çalışır?

OpenShift:

- On-premise (VMware, bare metal)
- AWS, Azure, GCP
- Hybrid & Multi-cloud

Ne zaman OpenShift seçilir?

- Kurumsal (enterprise) projelerde
- Güvenlik çok önemliyse
- Kubernetes yönetmek zor geliyorsa
- Red Hat ekosistemi kullanılıyorsa

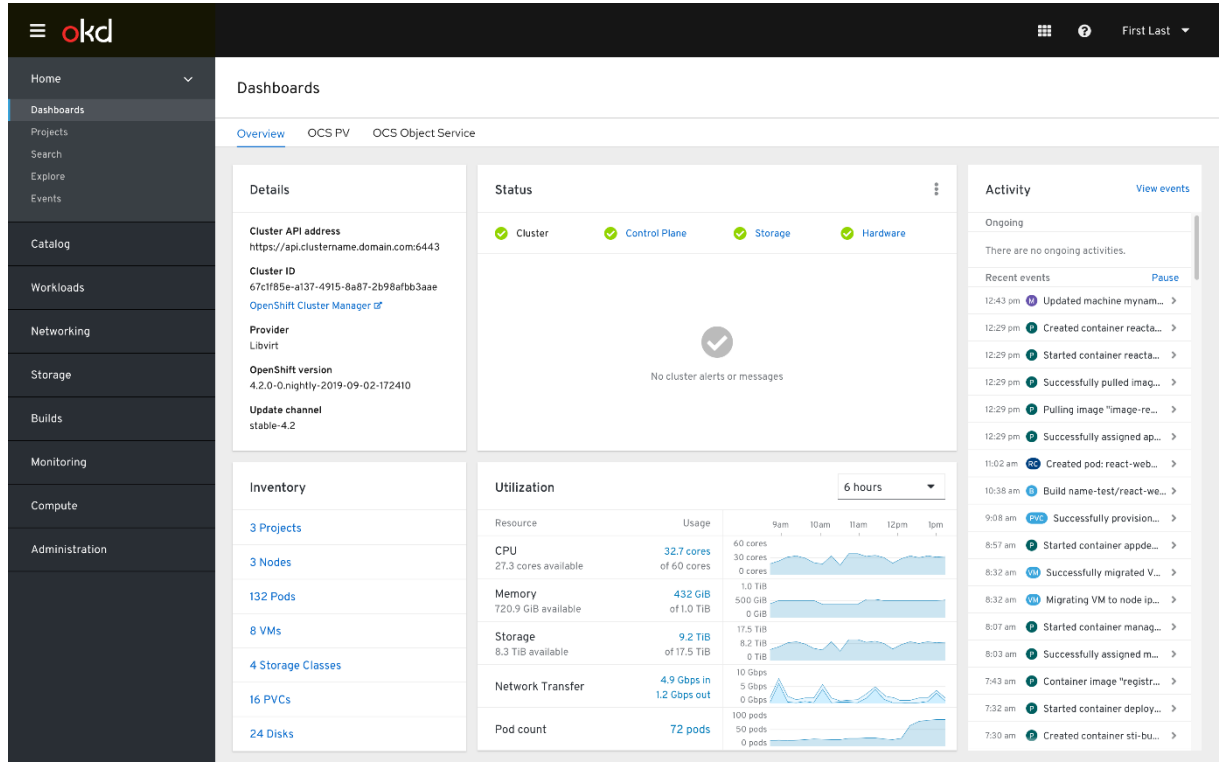
OpenShift'i Nasıl Çalıştırabiliriz?

Günümüzde Red Hat OpenShift çeşitli mekanizmalar ve formatlar aracılığıyla kullanılabilir:

- DevOps ekipleri bunu kendi veri merkezlerine "yerinde" kurabilirler.
- AWS, Azure, Google Cloud Platform ve IBM Cloud gibi büyük hiper ölçekli bulut sağlayıcıları, yönetilen Red Hat OpenShift kurulumları sunmaktadır.
- Geliştiriciler, Red Hat OpenShift Local (CRC veya "Kod Hazır Konteynerler" olarak da bilinir) kullanarak OpenShift'i iş istasyonlarında yerel olarak çalıştırabilirler.
- Ayrıca, test ve değerlendirme amacıyla Red Hat tarafından sunulan 30 günlük ücretsiz OpenShift kümesi deneme sürümünü de talep edebilirler

OpenShift Arayüzü

OpenShift Giriş Sonrası Ana Ekran (Overview)

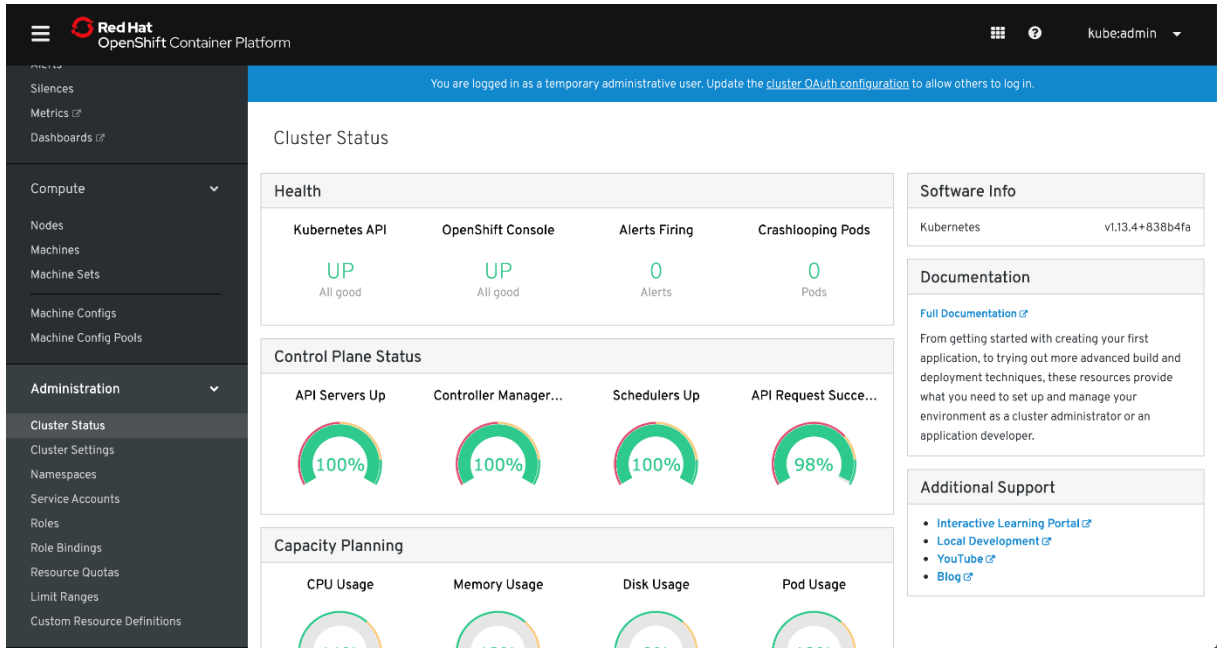


Bu ekran **cluster**'ın vitrinidir.

Burada şunları görürsün:

- Cluster **sağlıklı mı değil mi**
- Kaç **Node** var
- Kaç **Pod** çalışıyor
- CPU / Memory genel kullanımı

Sol Navigasyon Menüsü (Ana Menü)

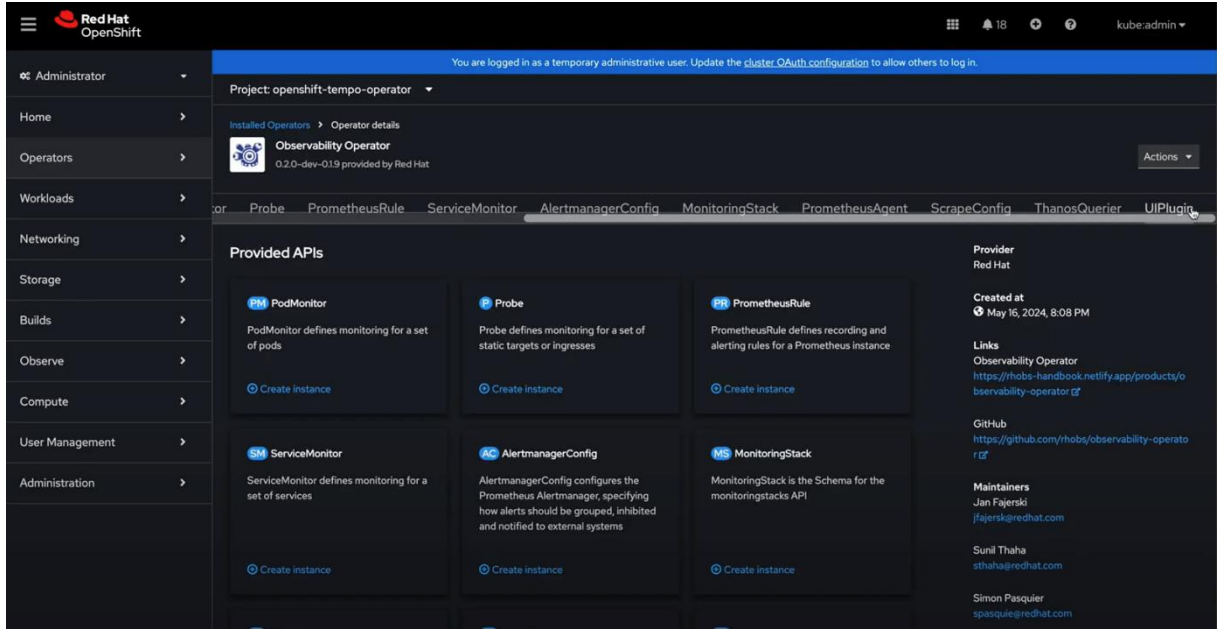


Sol menü OpenShift'in **kontrol panelidir**.

Ana başlıklar:

- **Home** → Genel durum
- **Workloads** → Pod, Deployment
- **Networking** → Service, Route
- **Storage** → Disk & PVC
- **Observe** → Log & Monitoring
- **Administration** → Node & cluster yönetimi

Developer / Administrator Perspektif Seçimi



OpenShift'te **iki farklı bakış açısı** vardır:

Developer:

- Uygulama deploy et
- Pod'lara bak
- Log izle

Administrator:

- Node yönet
- Operator kontrol et
- Cluster ayarları

Project (Namespace) Seçimi

Stateful Sets » world

world

app world name world

Details Environment Metrics Events

Status: Active
Replicas: 2 replicas

2 pods

Template

Containers

world

- Image: aosqe/hello-openshift
- Ports: 8080/TCP (web)
- Mount: volume1 → /var/lib/volume-test read-write
- Memory: 256 MiB limit

Volumes

volume1

Type: empty dir (temporary directory destroyed with the pod)
Medium: node's default

Pods

Name	Status	Containers Ready	Container Restarts	Age
world-1	Running	1/1	0	a few seconds
world-0	Running	1/1	0	a few seconds

There are no annotations on this resource.

OpenShift'te her şey **Project (Namespace)** altında çalışır.

Örnek:

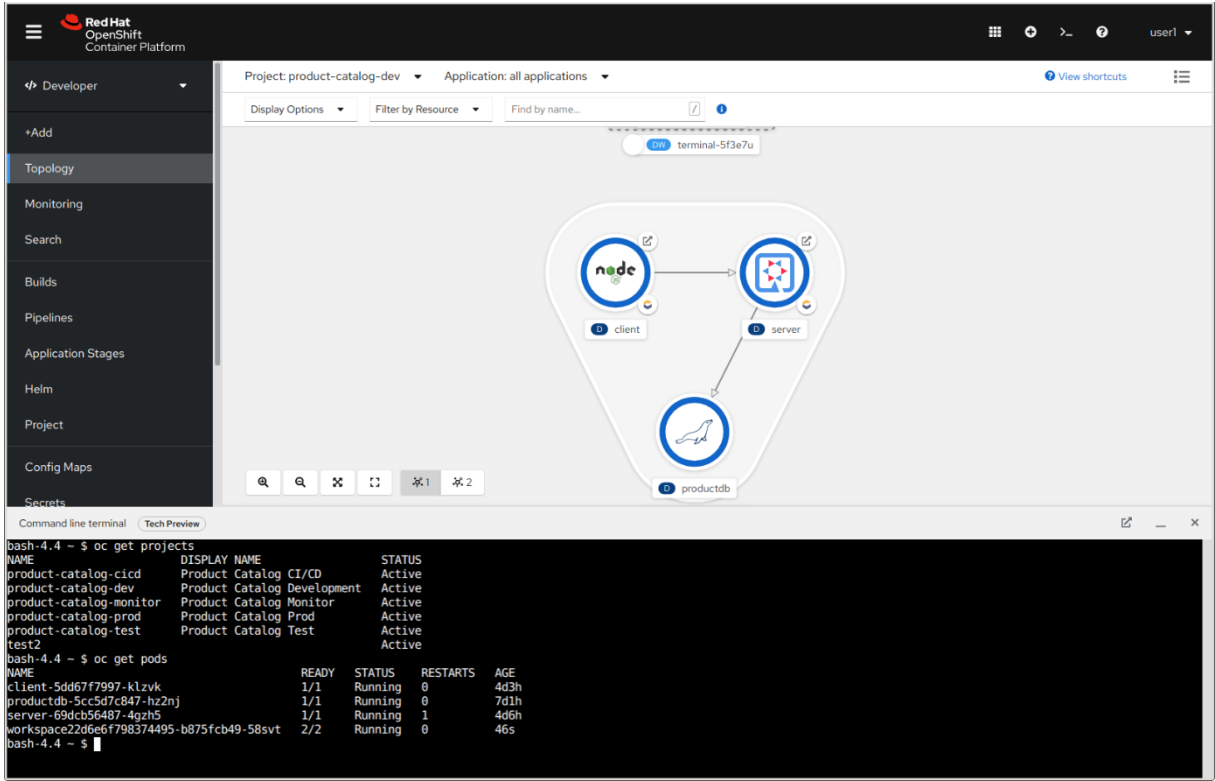
- dev-project
- test-project
- prod-project

Üstteki çubuk içinde **aktif proje adı** görünür (ör. My Project).

Buraya tıklayarak başka projeler arasında geçiş yapabilirsin.

Bu menü sayesinde UI'da hangi proje kapsamındayız belirlenir; diğer kaynaklar bu seçime göre filtrelendir.

Topology Görünümü (Uygulama Haritası)



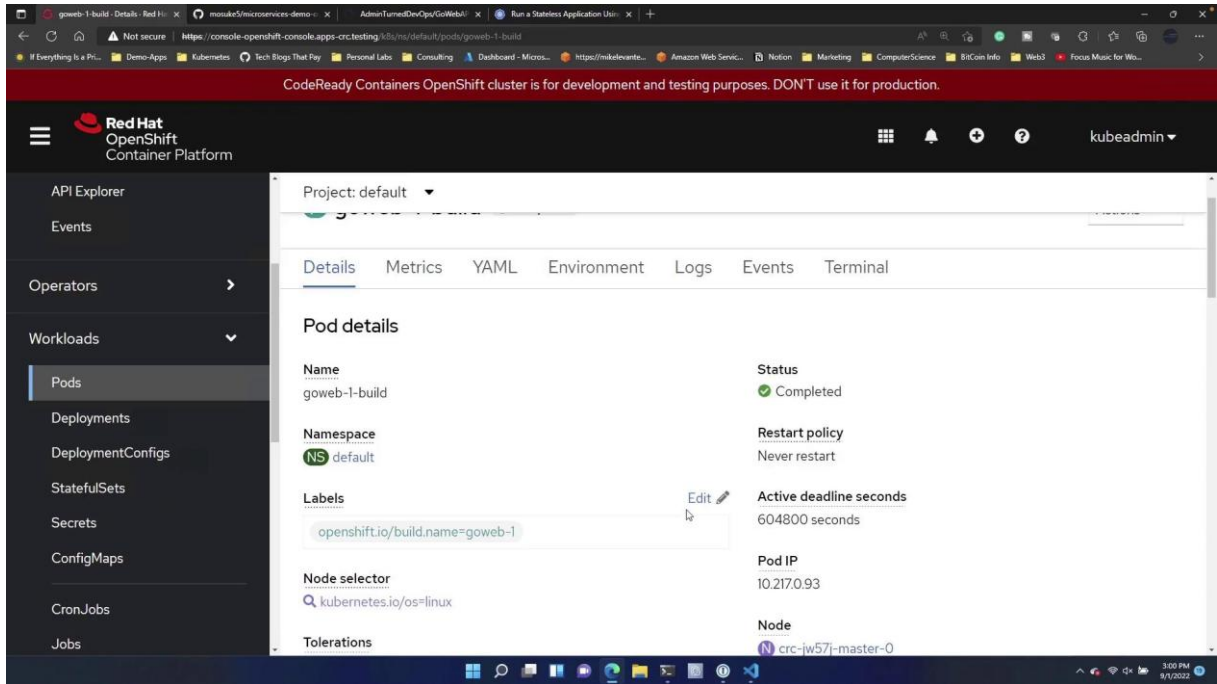
Bu ekran uygulamanın görsel haritasıdır.

Görünür:

- Deployment
- Pod'lar
- Service bağlantıları
- Route (URL)

“Uygulamam nerede, neye bağlı?” sorusunun cevabı

Workloads → Pods



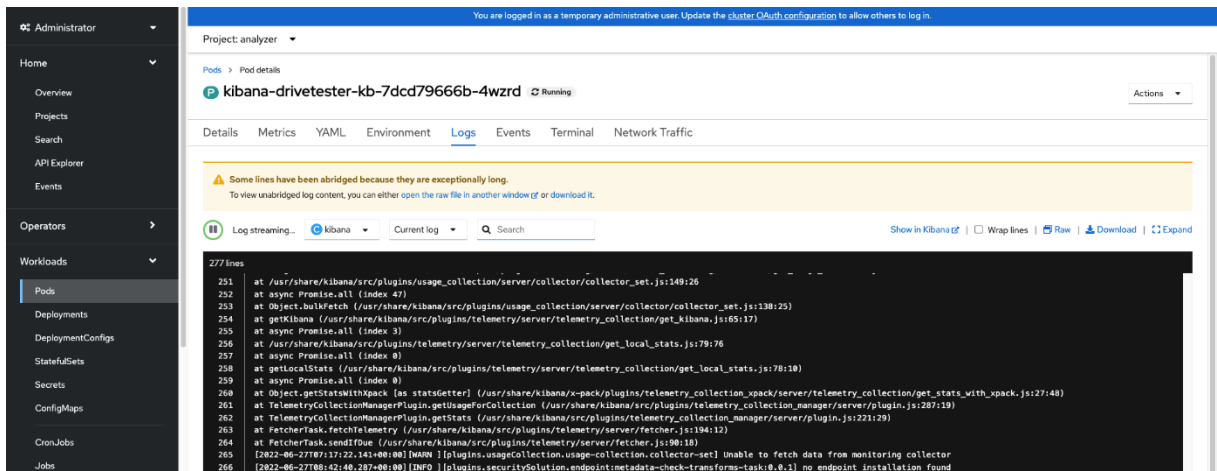
Pod listesi uygulamanın kalp atışı gibidir.

Burada:

- Pod Running mi?
- Kaç kere restart olmuş?
- Hangi node'da çalışıyor?

Hata durumunda ilk bakılan yer

Pod Detayı & Log Görünümü

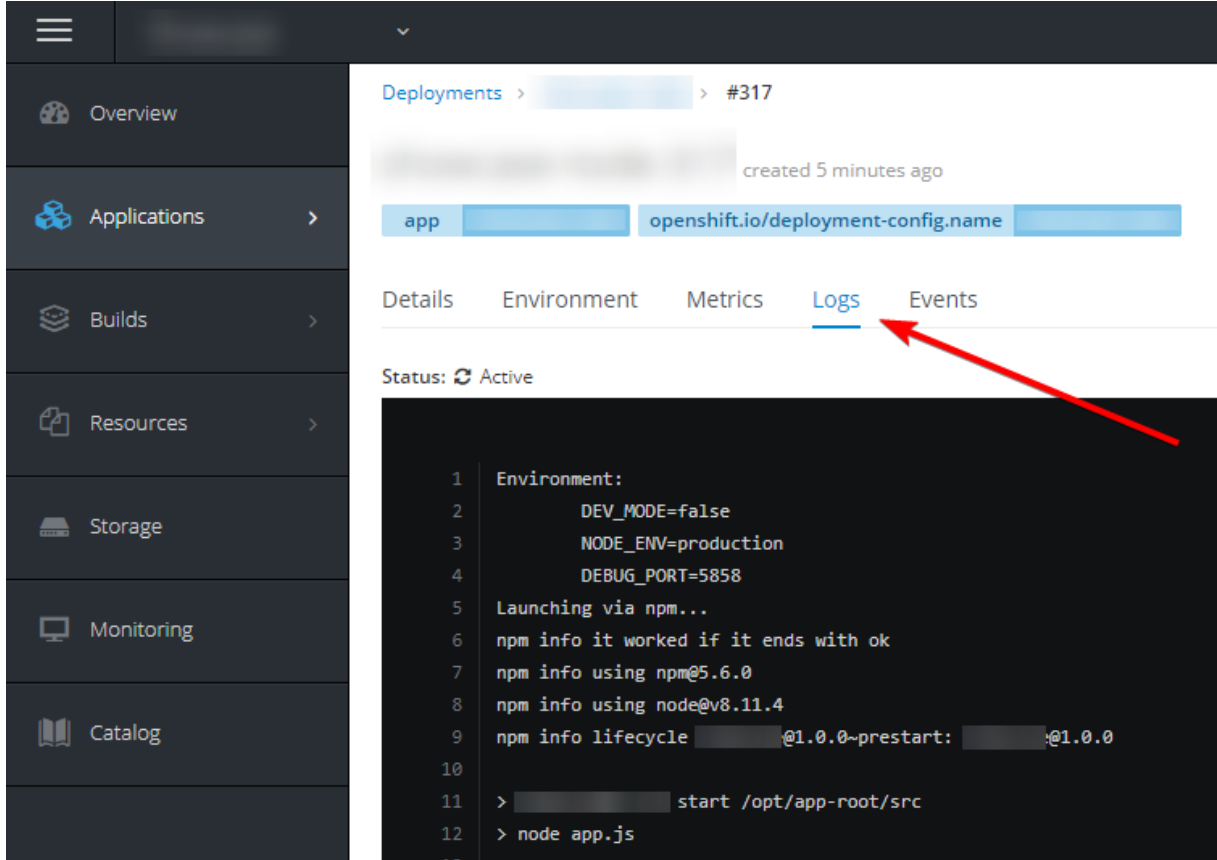


Bir Pod'a girince:

- Container log'ları

- Event’ler
- Restart nedeni

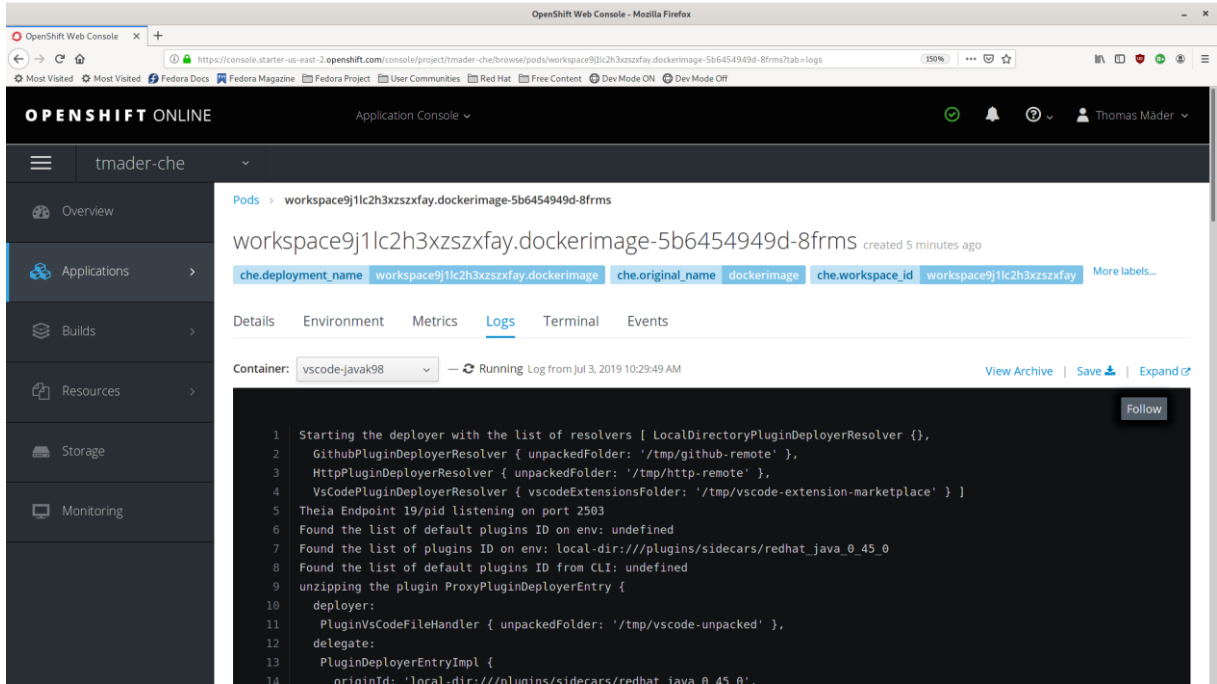
Log Sekmesinin Üst Bölümü



Üstte “Details”, “YAML”, “Environment”, “**Logs**”, “Events”, “Terminal” gibi sekmeler bulunur.

Logs sekmesi, konsolda çalışan uygulamanın anlık çıktısını gösterir — hata mesajları ya da bilgi logları gibi.

Pod Detayı & Log Örneği



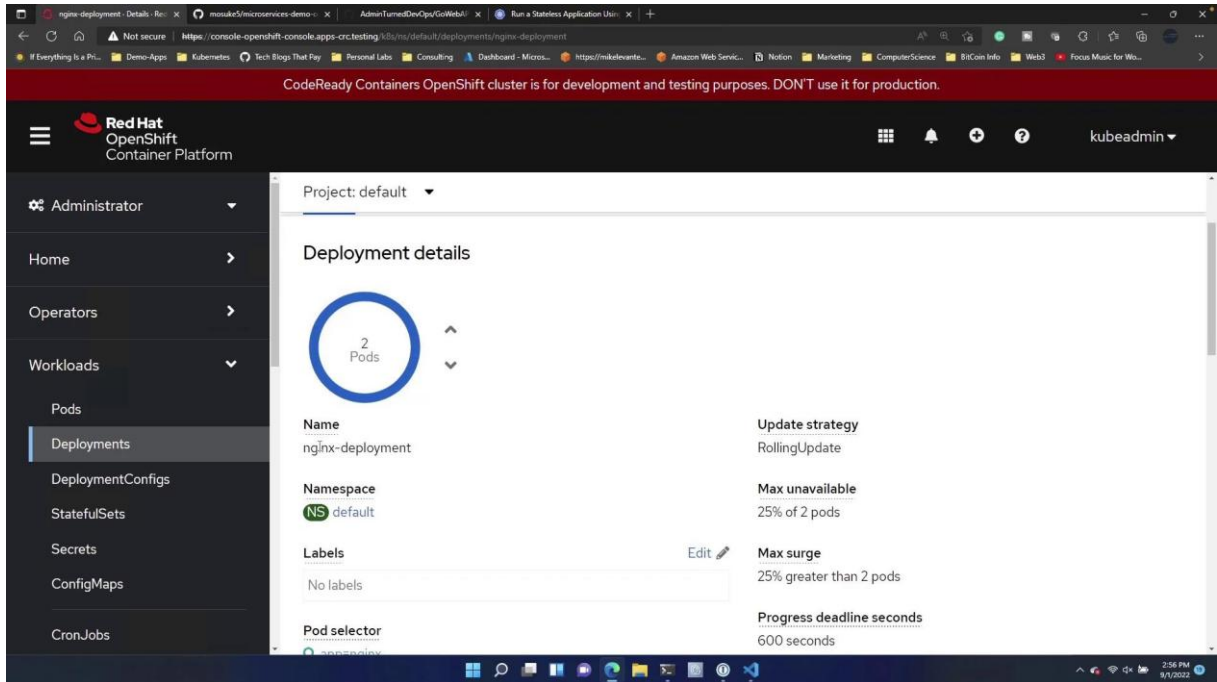
Burada Console'da hem **Pod detayları** hem de **Logs (konsol çıktısı)** bir arada görülür.

Log ekranında kaydırarak daha eski logları da görüntüleyebilir veya indirebilirsin.

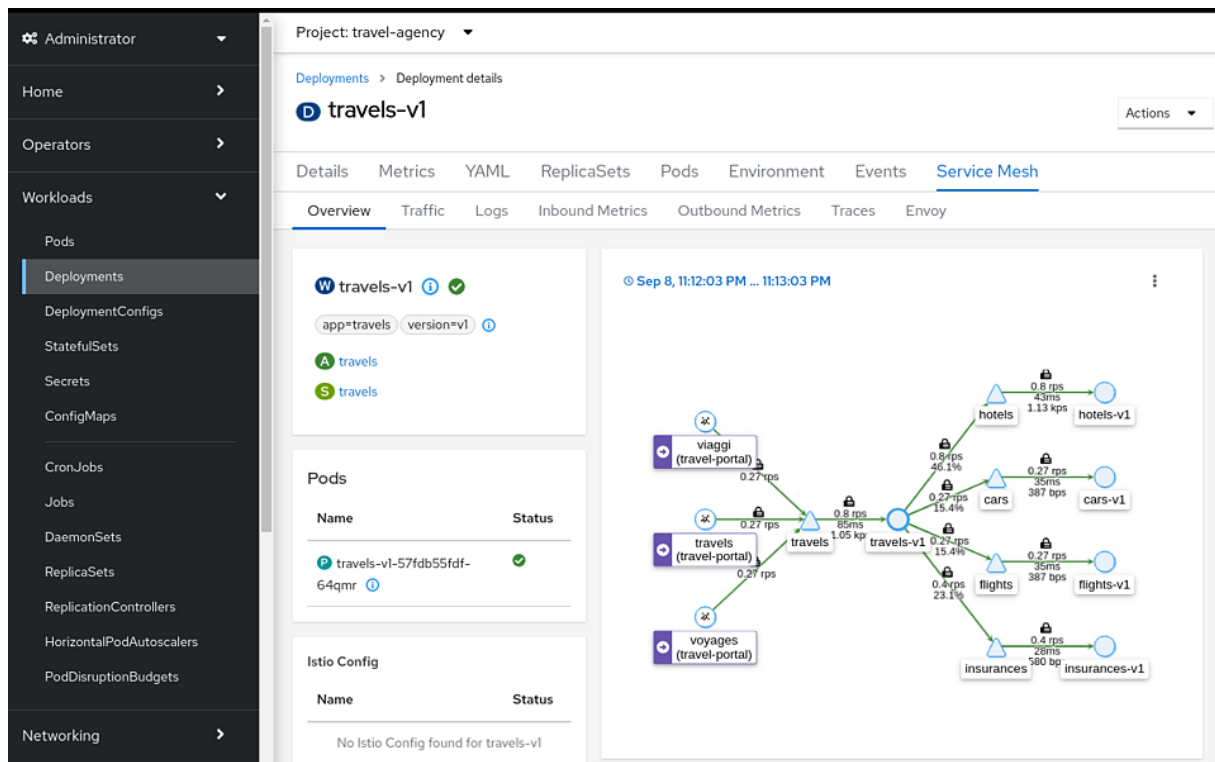
Amaç:

“Neden çalışmıyor?” sorusuna cevap

Deployment Yönetimi



Deployment ekranı **uygulama kontrol merkezidir**.



Yapılan işlemler:

- Scale (replica sayısı)
- Yeni versiyon deploy
- Rollback

Networking → Services

Administrator

Networking

Services

Routes

Services

Filter by name...

Service Name	Namespace	Type	Cluster IP	Ports	Age
frontend	prod	ClusterIP	10.128.1.10	80/TCP	2d
backend	prod	ClusterIP	10.128.1.20	8080/TCP	2d
database	prod	ClusterIP	10.128.1.30	5432/TCP	5d

Service:

- Pod'lar arası **load balancing** yapar
- Sabit IP / DNS sağlar

Pod ölür, Service kalır

Networking → Routes (URL)

Route = Uygulamanın **tarayıcıdan açılan adresi**

Örnek:

<https://myapp-prod.apps.cluster.local>

Ingress yerine OpenShift Route kullanır

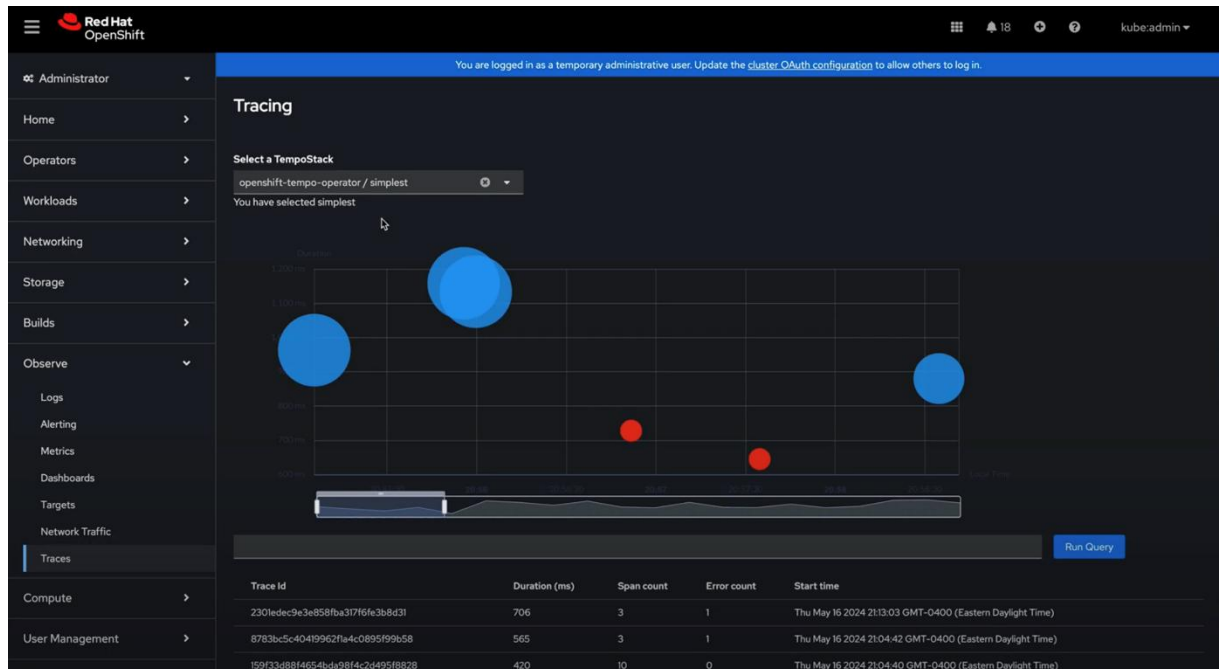
Storage → PVC

PVC:

- Disk talebi
- Veritabanı verisi
- Kalıcı storage

Pod silinir → veri silinmez

Observe → Monitoring



Burada:

- CPU
- Memory

- Network grafikleri izlenir.

Alt yapı:

- Prometheus
- Grafana

Performans ve kapasite takibi

Administration → Nodes

Cluster'daki tüm **node'lar** burada görünür.

Bilgiler:

- Node rolü
- Ready / NotReady
- Kaynak kullanımı

Altyapı yöneticisinin ana ekranı

Cluster Operators

OpenShift'in **kendi bileşenleri** burada izlenir.

Örnek:

- API Server
- Scheduler
- Monitoring
- Networking

Operator DOWN = cluster risk altında

Kaynaklar

<https://bulutistan.com/blog/openshift-nedir/>

<https://www.cenuta.com/blog/container-nedir-ozellikleri-ve-kullanim-olanlari/>

<https://www.linkedin.com/pulse/openshifte-giri%C5%9F-temel-kavramlar-ve-kullan%C4%B1m-senaryolar%C4%B1-a6wcf/>

<https://openshift.guide/getting-started/what-is-openshift.html>